

```
FLAG{CONFIDENTIAL_DATA:_PAYLOAD_TARGET_LOCATION=LAT:34.087_LON:-118.312_ALT:500}
```

```
CONFIDENTIAL_DATA: PAYLOAD_TARGET_LOCATION=LAT:34.087 LON:-118.312 ALT:500
```

```
from pathlib import Path
import zlib
import struct
```

```
# Change this path if needed
BASE = Path("pr_6_files")
```

```
drone_file = BASE / "dronecorp_flight_001.DAT"
payload_file = BASE / "intercepted_payload.dat"
```

```
# -----
# Part 1: Solve drone DAT file
# -----
```

```
enc = drone_file.read_bytes()
```

```
# zlib streams commonly start with 0x78.
# enc[0] XOR key = 0x78
drone_xor_key = enc[0] ^ 0x78
```

```
zlib_data = bytes(b ^ drone_xor_key for b in enc)
plain = zlib.decompress(zlib_data)
```

```
flag_start = plain.index(b"FLAG{")
drone_flag = plain[flag_start:].decode()
```

```
# Parse telemetry
magic = plain[:4]
version = plain[4]
embedded_key = plain[5]
```

```
flight_time = struct.unpack_from("<I", plain, 8)[0]
lat = struct.unpack_from("<d", plain, 12)[0]
lon = struct.unpack_from("<d", plain, 20)[0]
alt = struct.unpack_from("<f", plain, 28)[0]
```

```

distance = struct.unpack_from("<f", plain, 32)[0]

print("[+] Drone file decrypted")
print(f"  XOR key    : 0x{drone_xor_key:02x}")
print(f"  Drone flag  : {drone_flag}")
print(f"  Magic       : {magic!r}")
print(f"  Version     : {version}")
print(f"  Embedded key : 0x{embedded_key:02x}")
print(f"  Flight time  : {flight_time}s")
print(f"  Latitude     : {lat}")
print(f"  Longitude    : {lon}")
print(f"  Altitude     : {alt}m")
print(f"  Distance     : {distance}m")

```

```

# -----
# Part 2: Solve intercepted payload
# -----

```

```

payload = payload_file.read_bytes()

```

```

if payload[:4] != b"PAYL":
    raise ValueError("Payload does not start with PAYL header")

```

```

# After the first PAYL header, the next layer is XOR encrypted.
# We know the decrypted inner header should again start with PAYL.
outer_xor_key = payload[4] ^ ord("P")

```

```

layer1 = bytes(b ^ outer_xor_key for b in payload[4:])

```

```

if layer1[:4] != b"PAYL":
    raise ValueError("Failed to decrypt outer payload layer")

```

```

inner_encrypted = layer1[4:]

```

```

# Inner layer is repeating XOR with key PAYL
inner_key = b"PAYL"

```

```

secret = bytes(
    b ^ inner_key[i % len(inner_key)]
    for i, b in enumerate(inner_encrypted)
)

```

```
).decode()
```

```
print()
```

```
print("[+] Intercepted payload decrypted")
```

```
print(f"    Outer XOR key : 0x{outer_xor_key:02x}")
```

```
print(f"    Secret data   : {secret}")
```

```
# README says final flag format is FLAG{decrypted_data}
```

```
final_flag = "FLAG{" + secret.replace(" ", "_") + "}"
```

```
print()
```

```
print("[+] Final flag")
```

```
print(final_flag)
```